Ponto CLT — Como funciona

Atria ERP (azuz-erp) · Situação em 24/09/2026 · Fases 1 a 3 concluídas (cadastro, bater ponto e lembretes)

1. O que é

Uma aba dentro do próprio sistema onde os **funcionários CLT** registram entrada, saída para almoço, volta do almoço e saída. O gestor cadastra as jornadas e quem bate ponto. Quem esquece de bater recebe um **lembrete no sino** de notificações.

1. Gestor configura
Em *Configurações → Ponto CLT*, cria a jornada (ex.: seg a qui, 08:00–19:00, almoço 13:00–14:00) e cadastra os funcionários CLT.

2. Funcionário bate ponto
Na aba *Bater ponto*, um único botão registra a próxima marcação do dia, com comprovante (NSR).

3. Sistema lembra
A cada 5 minutos, quem passou do horário previsto + 15 min sem marcar recebe aviso no sino.

2. Regras de negócio

Regra	Como é garantida
Só CLT bate ponto	Apenas usuários com cadastro trabalhista ativo veem a aba e conseguem registrar (os demais recebem 403).
Horário do servidor	O horário vem do servidor, no fuso de Brasília. Alterar o relógio do computador não tem efeito.
Ordem das marcações	Entrada → Saída almoço → Volta almoço → Saída. A 5ª marcação do dia é recusada.
Marcação nunca é alterada	Um *trigger* no banco bloqueia qualquer UPDATE. Correções serão feitas por pedido de ajuste (fase 5).
Sem duplicidade	Clique duplo (< 1 min) é recusado. Cliques simultâneos geram uma única marcação (transação que trava o contador da empresa).
NSR (nº sequencial)	Cada marcação recebe um número único e crescente por empresa, com 9 dígitos, como no padrão oficial.
Lembretes	Um por marcação por dia, só em dias de jornada, sempre sobre a *próxima* marcação pendente. Tolerância configurável (padrão 15 min).
Horas do dia	Trabalhado = (entrada → saída almoço) + (volta almoço → saída). Jornada atual: 10h/dia, 40h/semana.

3. Arquitetura (SOLID)

O backend é um módulo NestJS separado em camadas. As regras não dependem do banco: a aplicação usa **abstrações** (portas) e a infraestrutura fornece as implementações. Isso permite testar tudo com dados em memória e trocar peças (ex.: e-mail no lugar do sino) sem mexer nas regras.

Camada	Responsabilidade	Principais arquivos
domain	Regras puras, sem banco: horários, ordem das marcações, fuso de Brasília, cálculo de horas, quando lembrar.	`work-schedule.ts`, `time-clock-entry.ts`, `work-date.ts`, `clock-reminder.ts`, `clock-time.ts`
application	Casos de uso e portas (abstrações de repositório, relógio e notificador).	`work-schedules.service.ts`, `employee-profiles.service.ts`, `time-clock.service.ts`, `clock-reminders.service.ts`, `ports/`
infrastructure	Implementações com Prisma, relógio do sistema, envio ao sino e tarefa agendada.	`prisma-*.repository.ts`, `system-clock.ts`, `bell-reminder-notifier.ts`, `clock-reminder.scheduler.ts`
presentation	Rotas HTTP, validação de entrada (DTOs) e formato das respostas.	`*.controller.ts`, `dto/`, `*.mappers.ts`

4. Onde encontrar o código

O quê	Caminho
Módulo do backend	`backend-atria/src/time-tracking/` (registrado em `app.module.ts`)
Permissão de gestor	`backend-atria/src/auth/constants/permissions.ts` → `timeclock:manage` (ADMIN; MASTER tem tudo)
Banco (schema `dev`)	`backend-atria/prisma/schema.prisma` e `prisma/migrations/20260924120000_add_time_tracking`, `20260924120100_add_time_clock_reminder_notification_type`
Tela do funcionário	`frontend-atria/app/(main)/ponto/page.tsx`
Tela do gestor	`frontend-atria/app/(main)/settings/ponto/page.tsx`
Componentes	`frontend-atria/components/time-tracking/` (cartão de bater ponto, marcações do dia, relógio, jornadas, funcionários)
Chamadas à API e hooks	`frontend-atria/services/time-tracking.service.ts`, `hooks/use-time-clock.ts`, `hooks/use-time-tracking-admin.ts`
Menu e notificações	`components/layout/navigation.ts`, `hooks/use-visible-nav-sections.ts`, `lib/notification-utils.ts`
Testes	Unitários: `backend-atria/src/time-tracking/**/*.spec.ts` · e2e: `backend-atria/test/time-tracking.e2e-spec.ts`

5. Banco de dados

Tabela	Conteúdo
`work_schedules` / `work_schedule_days`	Jornadas e horários de cada dia (em minutos desde 00:00; ex.: 08:00 = 480).
`employee_profiles`	Dados trabalhistas: usuário, regime CLT, admissão, jornada, ativo. Separado do `User` (login).
`time_clock_entries`	Marcações (somente inserção), com NSR, tipo, dia e horário.
`time_clock_counters`	Último NSR usado por empresa.
`time_adjustment_requests`	Pedidos de ajuste (estrutura pronta; fluxo na fase 5).
`time_clock_reminders`	Lembretes já enviados — impede repetição.

6. Rotas da API

Rota	Uso
`GET/POST/PATCH /time-tracking/work-schedules`	Gestor: listar, criar, editar, (des)ativar jornadas
`GET/POST/PATCH /time-tracking/employees`	Gestor: listar, cadastrar, editar, (des)ativar funcionários CLT
`GET /time-tracking/me/profile`	Usuário logado: saber se bate ponto (controla o menu)
`GET /time-tracking/me/clock/today`	Funcionário: marcações do dia, próxima marcação, horas trabalhadas
`POST /time-tracking/me/clock`	Funcionário: bater ponto (retorna horário e NSR)

7. Testes

- **87** testes unitários do backend passando (inclui um dia completo simulado com relógio controlado).
- **32** testes e2e do ponto contra a API real no `dev`: permissões, validações, cliques simultâneos, imutabilidade e lembretes. O teste limpa os próprios dados e nunca mexe em funcionários reais.
- Rodar: `npx jest src/time-tracking` e `npx jest --config test/jest-e2e.json test/time-tracking.e2e-spec.ts` (em `backend-atria`).

8. Pendências e próximos passos

> **Antes do uso real:** zerar o contador de NSR (os testes o avançaram). O comando já foi apresentado e será executado somente com autorização.

- **Fase 4** — Espelho do mês: histórico diário, horas trabalhadas, extras e faltas.
- **Fase 5** — Pedidos de ajuste com aprovação do gestor e visão do gestor (com exportação Excel).
- **Depois** — lembretes por e-mail/WhatsApp, aviso ao gestor, banco de horas, adicional noturno, feriados, AFD/AEJ.

Observação legal: a jornada de 10h/dia (seg–qui) caracteriza compensação de jornada e exige acordo escrito com cada funcionário — confirmar com o contador.